Hierarchy-Aware Context Compression for Coding Agents, and a Premature-Termination Failure It Induces

Andrew Lin
ahmlchannel2025@gmail.com

Abstract

Agentic coding systems that run a task step by step resend their entire transcript on every model call, so the cost of a run is dominated not by the work but by context that is paid for again at each step. We instrument a minimal step-wise coding agent and measure, across three task shapes and 63.4 million tokens, that input (context resend) accounts for 97.9% of all tokens, invariant across tasks (97.8–98.0%). We test a hierarchy-aware intervention: at each boundary between high-level goals, the transcript of completed goals is replaced by a short summary that preserves the task specification, the shared contract, the files on disk, and function signatures. On a code-maintenance task over twenty matched seeds, compression reduces total tokens by 54.2% (summary calls included), cheaper on every seed. It does not come free: global task success, judged by a hidden test suite independent of the agent's self-reports, falls from 20/20 to 18/20. The lost runs are not caused by missing information — in every failure the agent re-acquired the signatures it referenced after the reset — but by premature termination: the compressed context reads as finished and the model ends its turn without acting, even though its instructions, tools and plan are retained in the prompt. A continuation footer reduces the effect but does not remove it; it is a low-rate stochastic failure, and in two surviving cases the agent states that a sub-step is done, names the next sub-step as not yet done, and stops anyway. We report compression as a large, measurable saving with a specific and characterized cost, not as a free reduction, and we argue that context resend, not self-verification, is where step-wise agent spend actually goes.

---

1 Introduction

A coding agent that works autonomously decomposes a task into goals and sub-steps and acts on them in turn: it reasons, calls a tool, observes the result, and repeats. Because the underlying model holds no memory between calls, the transcript of everything done so far is the agent's only state, and it is resent in full on every call. The consequence is arithmetic rather than subtle: input grows with the length of the run, and a long task pays for its early steps many times over.

A common assumption in work on agent efficiency is that the waste to attack is redundant self-verification — the agent re-checking work it has already done. We find this misdirected. Measuring where tokens go across three task shapes, we observe that context resend, not verification, is 97.9% of all tokens, and that this fraction barely moves between a greenfield construction task and a maintenance task. Verification and recovery steps together account for single-digit percentages. The cost that dominates is structural: it is the price of being stateful through a stateless interface.

This suggests attacking the resend rather than the checking. We test a hierarchy-aware intervention: at each boundary between high-level goals, the verbose transcript of completed goals is replaced with a compact summary that keeps what later steps provably need — the specification, the shared contract, the files on disk, and the signatures of functions already written — and drops the per-step tool logs. Every step after a boundary then resends a summary in place of a transcript, and the saving compounds over the remainder of the run.

On a code-maintenance task over twenty matched seeds the saving is 54.2% of total tokens, summary calls included, and compression is cheaper on every seed. But success, measured by a hidden test suite that never sees the agent's own claims, falls from 20/20 to 18/20. We trace the lost runs and find the cause is not information loss. In each failure the agent had re-read the signatures it later referenced. The failure is that the run ends with the model declining to call a tool at all: the compressed context reads as a finished piece of work, and the model signs off. We confirm this directly from a surviving failure under a mitigation, where the agent states a sub-step is complete, names the next sub-step as incomplete, and still stops. We therefore report compression as a large saving with a specific, reproducible cost, and we locate that cost precisely, so that it can be addressed rather than hidden.

Section 2 describes the agent and the independent success measure. Section 3 relates the work to adaptive-compute and prompt-compression literature. Section 4 gives the tasks and protocol. Section 5 reports the context-cost measurement, the compression result, and the failure analysis. Sections 6 and 7 give recommendations and limitations.

2 Setup and Measures

The agent.
We use a minimal ReAct-style coding agent: a loop in which the model is given the task, the available tools (write a file, run a shell command), and the transcript so far, and returns either a tool call or a terminal reply. The loop executes the tool, appends the observation, and repeats until the model signals completion or a step limit is reached. The agent is deliberately simple, with a small tool set and no retrieval or planning machinery beyond an explicit plan it produces at the start; we report results for this agent and do not claim they transfer unchanged to production systems.

Forced step-wise execution.
So that plan structure and execution structure coincide, the agent is constrained to act on one plan sub-step per call; a write that attempts the whole solution at once is rejected and re-requested. A boundary is the first executed sub-step of a new high-level goal; every other sub-step is internal. This makes "at a boundary" a well-defined point at which to compress and a well-defined position at which to count failures.

Independent success.
The quantity we certify on is global success: whether the finished code passes a hidden test suite, judged by process exit code alone. This check never reads the agent's own reports of whether a step succeeded. The separation is the methodological core of the work: an agent that grades its own output cannot be trusted to report that compression preserved correctness, so the arbiter of correctness is held entirely outside the agent. Where we report the agent's self-reported step outcomes, we label them untrusted and do not use them to establish any claim.

3 Related Work

Adaptive test-time compute.
A line of work allocates compute adaptively at inference, by estimated difficulty or by uncertainty: budgeting reasoning tokens, stopping self-consistency sampling early, routing queries to cheaper or costlier procedures. Surveys of this area organize methods by how much compute to spend and how the decision is made. What this literature does not do, to our knowledge, is allocate by the structure of the agent's plan, or treat the resend of accumulated context as the cost to reduce. Our intervention is orthogonal: it does not

change how much the agent thinks, only how much history it re-reads.

Prompt and context compression.
Methods that compress prompts before they reach the model — dropping low-information tokens, or distilling long contexts into shorter ones — share our goal of paying for fewer input tokens. Our setting differs in that the thing compressed is an agent's own growing transcript, the compression happens at semantically meaningful points (goal boundaries) rather than by generic token scoring, and the quantity we protect is downstream task success under an independent check, not reconstruction of the original text. The failure mode we report — that compression changes the model's turn-taking behaviour rather than depriving it of information — is, we believe, not emphasized in that literature.

Agent evaluation integrity.
Recent benchmarks measure whether coding agents genuinely complete tasks or satisfy visible checks while failing held-out ones. Our independent test suite is in that spirit. We note, as a secondary finding, that across sixty runs we observed no case of an agent passing all of its own step checks while failing the global check; on these tasks the agent's self-reports tracked true success.

4 Experimental Setup

Tasks.
We use three self-authored Python tasks, each with a hidden pytest suite defining true success. *Pipeline* builds three modules that must agree on a shared data schema; its boundaries are module splits joined by a contract. *Calc* builds a stateful calculator in one file, where later operations depend on earlier state; its boundaries are capability groups joined by state rather than by file. *Refactor* changes existing, already-passing code to a new specification that touches several sites while keeping old behaviour; it is the maintenance shape, and the one on which we run the compression experiment. The three were chosen so that "boundary" means a different kind of seam in each, rather than one kind tested three times.

Protocol.
All experiments use a single small model for development economy; we state this as a limitation rather than a general claim. Each task is run over twenty seeds that vary the sample data but not the task. For the compression experiment, the compressed arm uses the same seeds as a previously banked baseline arm, so each compressed run is compared against an uncompressed run on matched input. Compressed token totals include the cost of the summary-generating calls themselves, which we report separately rather than exclude.

Rates and intervals.
Where we report a failure rate we give it as failures divided by the steps that exist with that tag, with a 95% Wilson interval, because internal steps greatly outnumber boundary steps and raw counts mislead. We read intervals before point estimates: when two intervals overlap we treat the difference as not established, however far apart the central values appear.

5 Results

5.1 Where the tokens go.
Across all three tasks and 63.4 million tokens, input — the context resent on each call — is 97.9% of total token spend, and output is 2.1%. The fraction is nearly constant across task shapes: 98.0% on pipeline,

97.8% on calc, 97.8% on refactor. Within a run, input per step climbs from roughly five thousand tokens early to tens of thousands late, as the transcript the model must re-read accumulates. Steps spent on recovery from the agent's own mistakes account for single-digit percentages of tokens. The cost that an efficiency effort should target is therefore the resend of accumulated context, not the frequency of verification.

5.2 Failure location does not generalize.

We had hypothesized that failures concentrate at goal boundaries. They do not, as a general matter. The ratio of boundary to internal failure rate is 0.45 on pipeline and 0.00 on calc — failures are internal, not at the seams. On refactor the ratio is 1.11 by the objective measure, in the hypothesized direction, but the boundary and internal confidence intervals overlap, so the effect is not established; the self-report channel is suggestive (ratio 1.98) but is the untrusted one. We report boundary-concentrated failure as unsupported for construction tasks and, at best, an unconfirmed hint for maintenance tasks that our budget did not let us settle.

5.3 Compression: a large saving with a cost.

On refactor over twenty matched seeds, boundary compression reduces mean total tokens per run from 809,254 to 370,373, a 54.2% reduction, with the summary calls (3.5% of the compressed total) included; every one of the twenty seeds is cheaper under compression, by 30% to 74%. The per-step input curve changes shape as intended: instead of climbing monotonically it drops back at each boundary.

The saving is not free. Global success, by the independent suite, falls from 20/20 in the baseline to 18/20 under compression. Because the arms share seeds, this is a within-input comparison: two specific seeds that pass uncompressed fail when compressed. We do not report the saving as being at equal success, because it is not.

5.4 The failures are premature termination, not lost context.

The lost runs end for the same mechanical reason: the model returns a terminal reply that contains no tool call, so the run stops with the task unfinished. This end state occurs under compression and never in the twenty baseline runs, so compression induces it. The cause is not lost information, and the structure of our agent makes this precise. Compression clears only the conversation transcript; the system instructions and the tool definitions are rebuilt and re-sent on every call, compressed or not, and the task specification and full plan are retained verbatim. So the model still holds its instructions and its objective; what the reset removes is only the step-by-step history of completed goals. In every failure traced, the agent had also re-acquired the specific signatures its final steps referenced. What the compressed context changes is not what the model knows but how the conversation reads: a block of completed-goal summaries presents as a finished piece of work, and the model responds with a closing remark instead of the next action.

We hard-coded a mitigation into the compression itself: every reset block ends with an explicit continuation footer stating that the task is not complete, listing the remaining goals, and instructing the model that its next action must be a tool call for the current sub-step. We also logged the final no-tool reply so the behaviour could be read directly. The footer reduces the failure but does not remove it: across two separate re-run batches of the three originally failing seeds, one run still terminated early in each, and the seed that fails moves between batches rather than staying fixed — the failure is a low-rate stochastic event under compression, not a property of particular inputs. The two surviving failures are the clearest evidence of the mechanism, and they agree. In one, the agent writes that a sub-step adding a normalization function is done, states that it has not yet run the tests and has not yet wired the function into the parser, names that

as the next sub-step — and stops. In the other, it writes that a sub-step adding a per-category aggregation helper is done, states that it has not yet wired it into the summary, names that as the next sub-step — and stops. In both, the model holds the context, is correct about the remaining work, states it explicitly, and ends the turn anyway, with a continuation instruction present in the prompt. The failure is behavioural, and it is robust to being told not to do it.

6 Recommendations

1. Target context resend, not verification. On these tasks, verification and recovery are a few percent of tokens and context resend is almost all of it; efficiency effort spent on reducing self-checking is spent in the wrong place.
2. Measure compression against an independent success check, with summary-generation cost included. A token saving reported without an outside arbiter of correctness, or with the compressor's own calls excluded, overstates the result.
3. Treat compression's cost as behavioural, not informational. Because the failure is premature termination rather than lost information, the fix is in how the compressed context is framed — making the remaining work explicit — not in keeping more content.

7 Limitations

The experiments use one small model, so the magnitude of both the saving and the termination failure may differ for larger models; a stronger model may be less prone to signing off on a compressed context, which would change the trade-off. The compression result is on one task shape (maintenance) over twenty seeds; the other two tasks were used to test failure location, not compression. The 54.2% saving is established, but "at what success rate" is not: the gate failed at 18/20, and we lacked the budget to run the twenty-seed confirmation that would measure whether the footer mitigation restores parity. The footer result rests on two small re-run batches and establishes that the failure recurs, is stochastic, and is mechanical, but not the rate at which it occurs or is mitigated. The tasks are self-authored rather than drawn from an established benchmark, which aids control over where boundaries lie but limits external comparability; confirming the context-cost fraction and the termination effect on a public agent benchmark is the natural next step. Finally, the boundary-failure-location question on maintenance tasks remains open: the point estimate is in the hypothesized direction but the intervals overlap, and resolving it needs more runs than were available.